

**Solutions to FIT3080 Applied Class on Problem Solving as Search:
Irrevocable and Adversarial Search**

Exercise 1: Irrevocable Search Algorithms

- (a) What happens in Simulated Annealing if the temperature remains at ∞ at all times? What happens if the temperature starts at 0?

SOLUTION:

- If the temperature remains at ∞ , the algorithm always accepts a new state regardless of whether it is better or worse. Also, the algorithm will go into an infinite loop. The ∞ temperature pertains to worse states. If we reach that stage in the algorithm, a worse state will be accepted with probability 1. However, Best-So-Far will keep only the best state.
 - If the temperature starts at 0, the algorithm will perform only one iteration, and return the best of the initial state or new state. Best-So-Far will keep only the best state.
- (b) A genetic algorithm is to be used to evolve a binary string of length n containing only 1s. The initial population is a randomly generated set of binary strings of length n .
- i. Propose a suitable fitness function for this problem.
 - ii. Calculate the probability of selecting each of the following strings according to your function.

00110001
 01011101
 11101111

SOLUTION:

- i. Sum of the genes
- ii.

00110001	$3/\{3+5+7\} = 0.2$
01011101	$5/\{3+5+7\} = 0.333$
11101111	$7/\{3+5+7\} = 0.467$

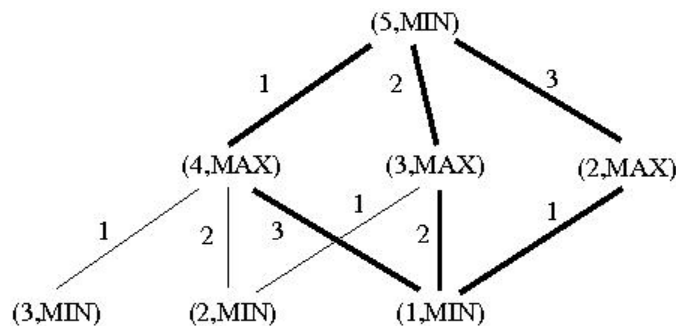
Exercise 2: Game Graphs

The game *nim* is played as follows: Two players alternate in removing one, two or three coins from a stack initially containing 5 coins. The player who picks up the last coin loses. Show by drawing the game graph, that the player who has the second move can always win. Can you think of a simple characterization of the winning strategy?

SOLUTION:

The characterization of the general winning strategy is to leave the opponent with 1, 5, 9, 13, ..., $4n + 1$, ... coins. You can figure this out from the solution below by trying 6, 7, 8, 9 coins (if MAX has 6, 7 or 8 coins, they can make sure MIN is left with 5 coins and lose, so the next winning number for MAX is to leave MIN with 9 coins, and so on).

The following diagram contains the game graph, where the winning paths are highlighted. Note that for **every** move that MIN makes, MAX must have **one** winning move.



Exercise 3: Minimax and α - β Search

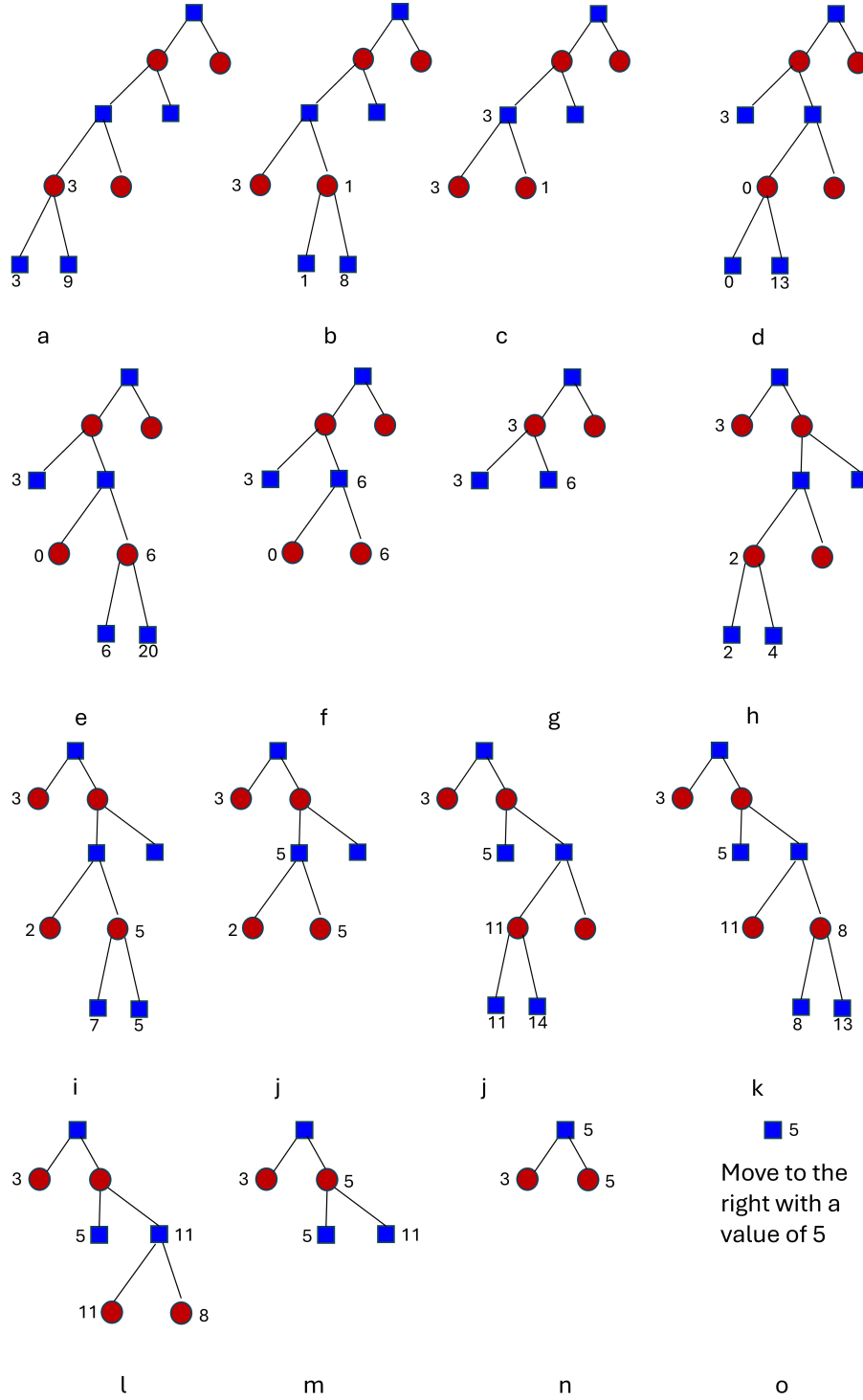
Consider a hypothetical game with branching factor 2. It is MAX's turn to play. They are able to evaluate a position 4 steps in advance. The following is a list of the values of the positions at the bottom of the game tree, should they ever need to be evaluated:

node #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
e(n)	3	9	1	8	0	13	6	20	2	4	7	5	11	14	8	13

- Draw a step-by-step game tree, as evaluated by MAX using the minimax procedure with DFS-style exploration.
- Draw a step-by-step game tree, as evaluated by MAX using the α - β procedure with DFS- or Backtrack-style exploration, so that only the expanded nodes appear in your diagram. Indicate clearly the α and β value of each node.
- How many α cut-offs and β cut-offs have been performed?
- What is the best move for MAX and what is its backed-up value?

SOLUTION:

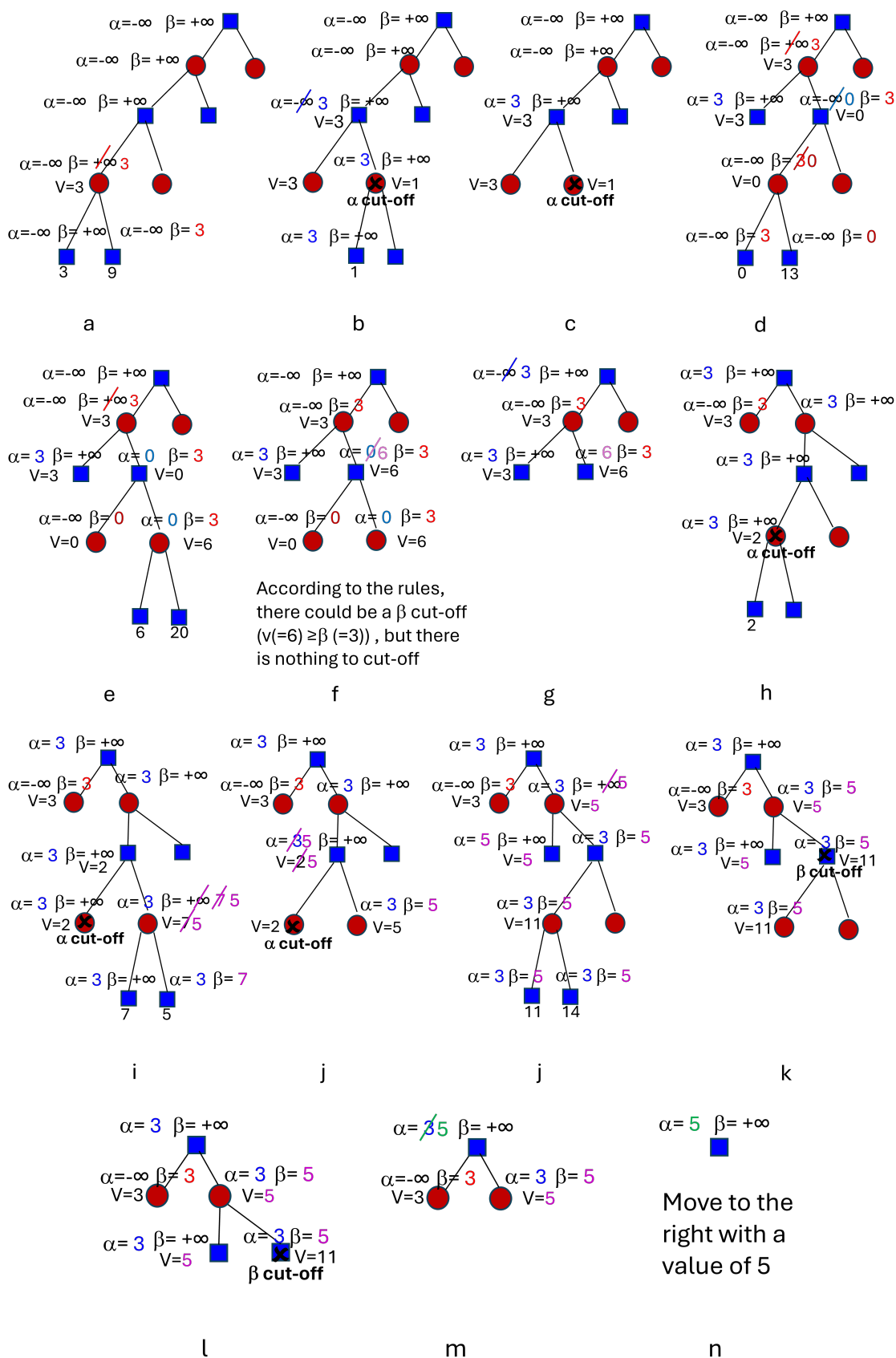
- The minimax game search tree produced by DFS-style exploration is as follows.



(b) Let us first revise the concepts:

- The α -value of a MAX-node is the value of the best (highest value) choice we have found so far at any choice point along the path for MAX.
- The β -value of a MIN-node is the value of the best (lowest value) choice we have found so far at any choice point along the path for MIN.
- α cut-off occurs if $v \leq \alpha$; β cut-off occurs if $v \geq \beta$.

The game search tree produced by DFS-style exploration is as follows. In DFS style, the children are added to the tree, but only evaluated if necessary. In Backtrack style, each child is added in turn, as necessary.



- (c) There are 2 α cut-offs, and 1 β cut-off.
- (d) The best move is **to the right** with value 5.

Exercise 4: Minimax Algorithm (Optional, Python coding)

Consider the complete game tree for Exercise 3. Using the outcomes for the 16 terminal nodes in `lab4_minimax_template.py` on Moodle, implement the minimax algorithm to find the first move if playing as Max.

SOLUTION:

See the code file `lab4_minimax_solution.py` on Moodle.